FLEETLENS USER GUIDE

Set up Fleetlens and start reading your agent fleet.

A practical first-run guide for the local dashboard, usage daemon, and optional Team Edition.

LOCAL EDITION	TEAM EDITION
One machine. Private by default. Reads local coding-agent session files and keeps the dashboard on localhost.	Shared visibility. An admin runs the server and teammates pair their local Fleetlens daemons to it.

For Fleetlens 0.16.4

The shortest path is: install the CLI, run `fleetlens start`, then open the URL it prints. You do not need to create an account for the local edition.

QUICK START

```
npm install -g fleetlens
fleetlens start --open
```

Which path should I use?

Start with the Local Edition if you are exploring Fleetlens or working alone. Choose Team Edition when multiple people need a shared dashboard and project-level rollups.

1. Before you start

Fleetlens is a local-first reader and analytics layer. The local dashboard does not require a database, a hosted account, or a project import step. It discovers the session files that your coding agents already write.

Prerequisites

- Node.js 20 or newer and npm. The published CLI is the easiest install path.
- At least one supported agent with local session history. An empty machine still starts successfully; the dashboard will simply show no sessions yet.
- For plan utilization, keep the relevant agent logged in. Claude Code usage is read through its existing OAuth credential; other providers expose their own local or configured usage sources.
- For Team Edition, an admin needs a deployed server URL and an invite or device token for each member.

What Fleetlens can read locally

Source	Local data	What appears
Claude Code	`~/.claude/projects`	Sessions, transcripts, usage, projects, PR signals
Codex	`~/.codex/sessions`	Sessions, transcripts, projects, local usage when available
Gemini CLI	`~/.gemini/tmp`	Session analytics and detail
Antigravity	`~/.gemini/antigravity-cli`	Session analytics and detail
Cowork	Local Cowork mirror	Session analytics and detail
Grok Build	`~/.grok/sessions or GROK_HOME`	Sessions and weekly usage when logged in
Z.ai	Configured API key	Plan usage only; no local transcript source

> **Privacy baseline**
> Local transcript files stay on the machine. The usage daemon still makes the provider requests required for plan meters. Optional AI features invoke the local claude CLI rather than sending raw transcripts to a Fleetlens service.

2. Install and open the local dashboard

The first-run path takes a minute or two and is safe to repeat.

1

Install the published CLI

Run the global install from a terminal. The npm package contains the CLI, parser, usage worker, and bundled Next.js dashboard.

TERMINAL

```
npm install -g fleetlens
fleetlens version
```

2

Start Fleetlens

Start launches the local web server and the background usage daemon together. Add --open if you want Fleetlens to launch your browser.

TERMINAL

```
fleetlens start
# or
fleetlens start --open
```

3

Open the printed URL

The default address is http://localhost:3321. If you selected another port, use the URL printed by the CLI.

4

Run an agent session

Fleetlens reads the agent's local files as they change. Refreshing is normally unnecessary because the dashboard listens for file events and refreshes its server-rendered data.

If you only want the web server

Use `fleetlens start --no-daemon`. You can later manage the worker independently with `fleetlens daemon start`.

What the local dashboard looks like

The first screen gives you a compact read on sessions, agent time, tools, concurrency, code changes, cost, and daily activity. The numbers below come from a sanitized fixture used only for this guide.

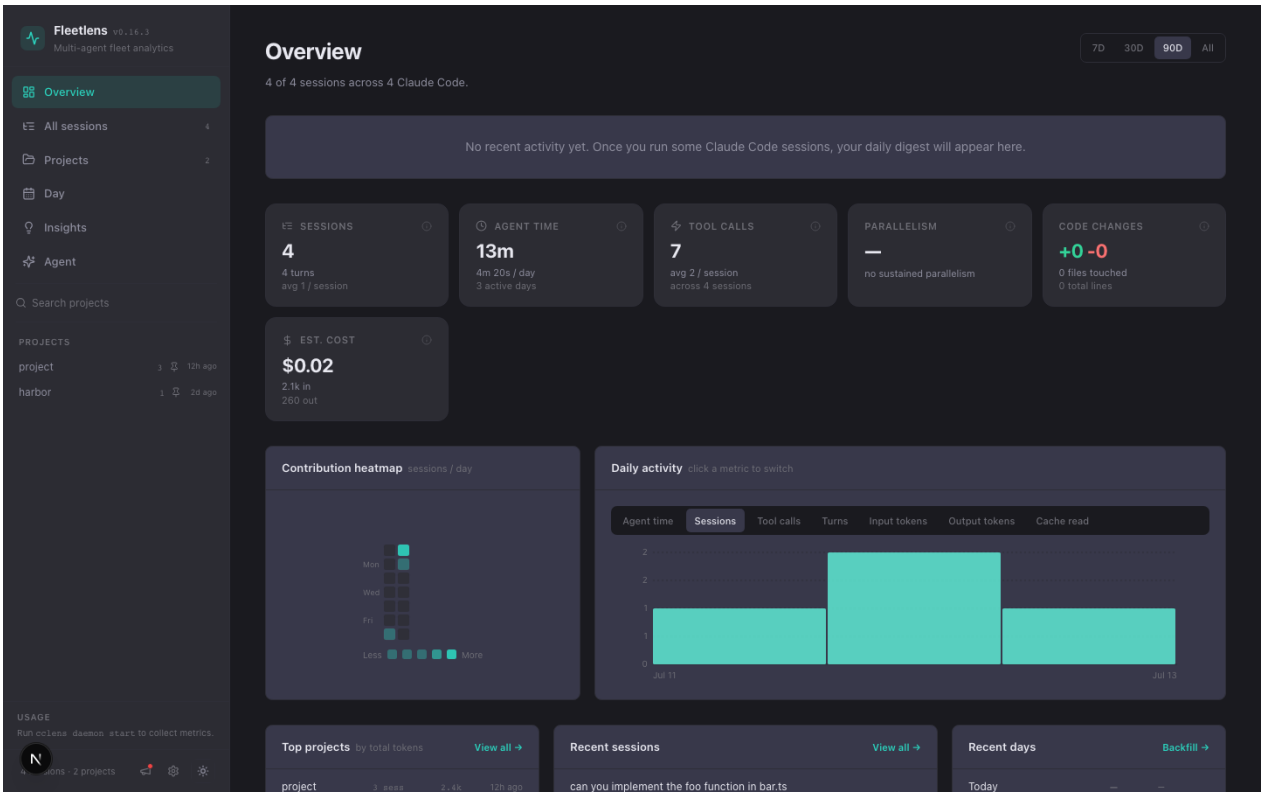

Figure 1. Overview with synthetic local fixture data. No personal transcript or project names are used.

3. Read the dashboard

Fleetlens normalizes different agent transcript formats into one session model. The dashboard can therefore compare sessions, projects, agent time, tool calls, turns, tokens, code changes, estimated cost, and concurrency in one place.

Primary pages

Page	Use it for
Overview /	Headline metrics, heatmap, daily activity, projects, recent sessions
All sessions /sessions	Search, filter, sort, and open individual transcript views
Projects /projects	Roll up activity by canonical project; worktrees fold into the parent repo
Day /day	Inspect daily activity, timeline, and concurrency bursts
Insights /insights	Read day/week/month digests when entries and AI features are enabled
Agent /agent	Ask questions over local session history and create handoff prompts
Usage /usage	Review historical plan utilization by agent
Settings /settings	Manage auto-start, menu bar widget, Z.ai credentials, and AI features
Team /team	Pairing status, sync selection, last push, and data boundary

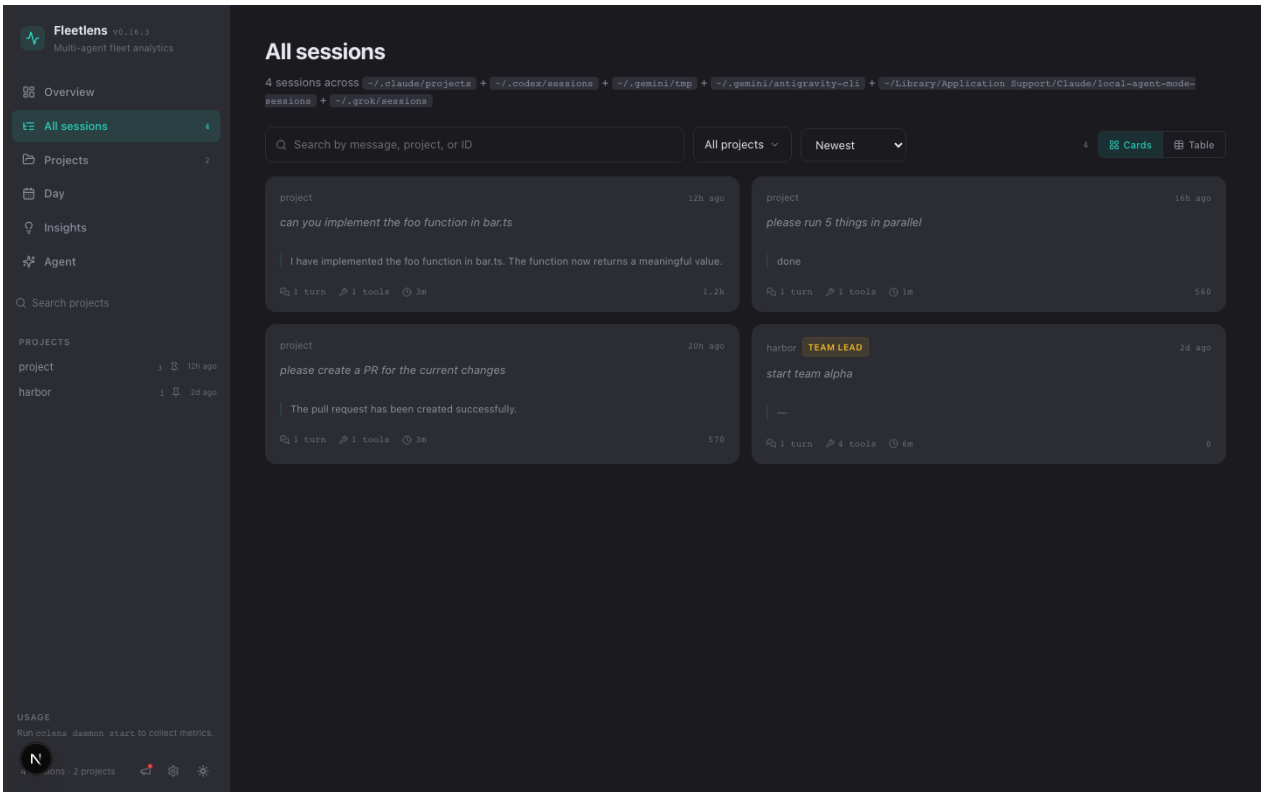

Figure 2. All sessions view: search, project filter, card/table toggle, and session-level metrics.

Agent time is active event time, not wall-clock time. Long gaps between transcript events are treated as idle, so the headline duration is more useful for understanding actual agent work.

4. Usage tracking and the background daemon

The daemon is a detached process that keeps plan snapshots in Fleetlens's local state directory. The dashboard's usage sidebar shows the latest snapshot; the Usage page charts historical snapshots and cycle boundaries.

Useful checks

TERMINAL

```
fleetlens status
fleetlens daemon status
fleetlens daemon logs
fleetlens usage
fleetlens usage --history
```

State files

Path	Purpose
`~/.cclens/pid`	Local web-server PID and port
`~/.cclens/daemon.pid`	Usage daemon PID
`~/.cclens/usage.jsonl`	Append-only usage snapshots
`~/.cclens/daemon.log`	Recent daemon and sync messages
`~/.cclens/entries/`	Day-scoped perception entries used by digests
`~/.cclens/digests/`	Saved day, week, and month digest artifacts

> **No usage snapshot yet?**
> The dashboard can still analyze transcripts. Check `fleetlens daemon status`, make sure the provider CLI is logged in, then inspect `fleetlens daemon logs`. Agents without a structured usage endpoint will not show a plan meter.

Optional macOS conveniences

- Settings can install the native menu bar widget when the bundled widget is present.
- The CLI can install daemon auto-start with fleetlens autostart install, or the settings page can enable it.
- The daemon also drives perception sweeps and, when enabled, backfills digest work in the background.

5. Optional Team Edition

Team Edition is a self-hosted Fleetlens server for shared rollups. The local CLI remains the source of truth for raw sessions; each paired machine chooses what project aggregates to sync.

Admin flow

- Deploy the Team Edition with the Railway template, Google Cloud installer, Docker Compose, or the AWS Terraform module.
- Open the server URL and create the first account. The first account becomes the admin of the first team.
- Create an invite or device token from the team server's settings area and send it to each teammate.

Member flow

TERMINAL

```
fleetlens team join https://your-fleetlens.example <invite-token>
fleetlens team status
fleetlens team sync
```

The join command opens the browser setup flow. Finish onboarding to select the projects that may sync. For a non-interactive first sync, use `fleetlens team join <url> <token> --no-browser`.

Common team commands

Command	Result
`fleetlens team status`	Pairing state, selected projects, and last sync
`fleetlens team sync`	Push unsynced days immediately
`fleetlens team backfill`	Re-upload local usage history for the shared dashboard
`fleetlens team logs`	Show recent team-related daemon messages
`fleetlens team leave`	Unpair and stop team syncing

Pairing starts from the Team page

Before pairing, the page gives the exact CLI command shape and reminds you that the admin must provide the token. After pairing, this page becomes the member's control surface for sync selection and status.

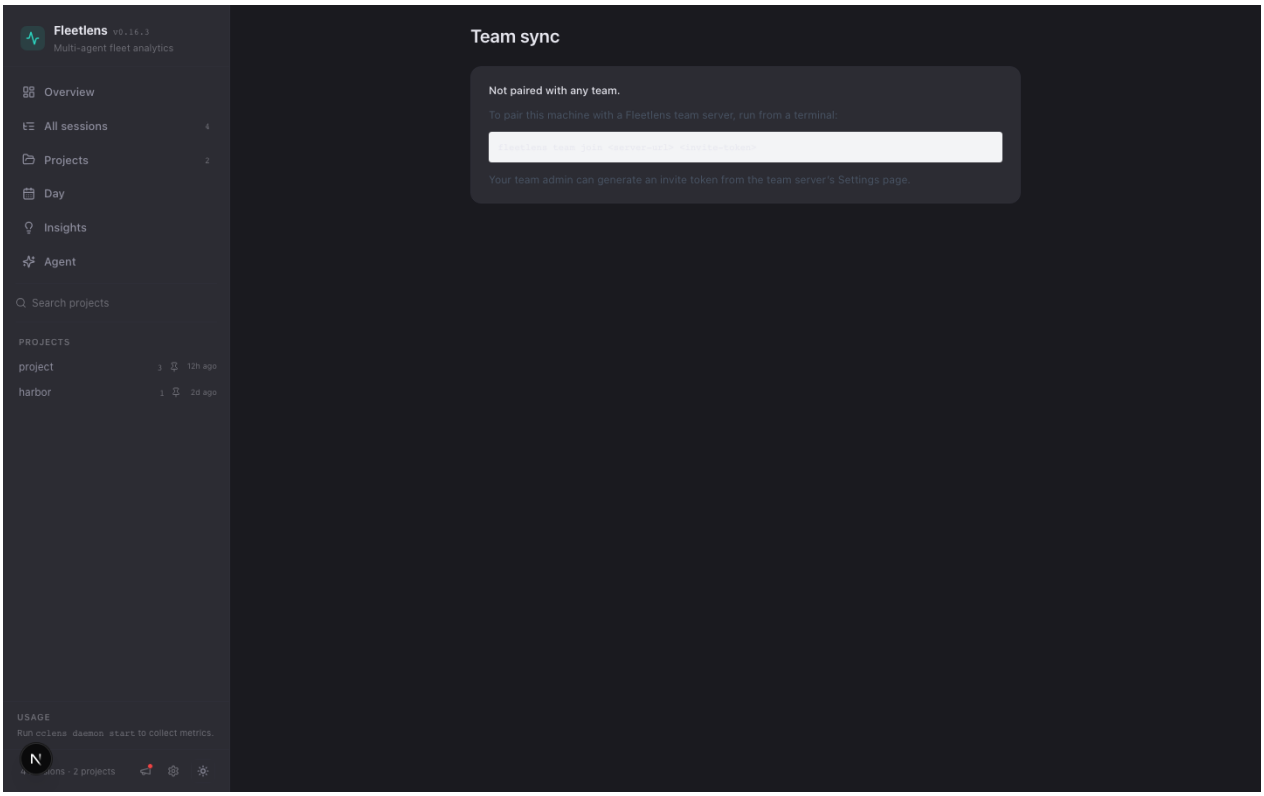

Figure 3. Team sync screen before pairing. After pairing it shows server, team, project selection, and last push state.

6. Understand the privacy boundary

Fleetlens has two deliberately different operating modes. Keeping the boundary visible helps teams decide what to deploy and what to sync.

Data	Local Edition	Team Edition sync
Raw transcripts	Read locally by the parser	Not uploaded
Prompts and assistant responses	Remain on the machine	Not uploaded
Absolute paths and file contents	Used locally for parsing and signals	Not uploaded
Daily metrics	Computed locally	Shared for selected projects
Rich project rollups	Computed from local entries	Shared as labeled aggregates
Usage snapshots	Stored in ~/.cclens/usage.jsonl	Shared for the paired member
AI enrichment	Runs through the local claude CLI when enabled	Enriched extras may be included in rollups

> **Project selection is member-controlled**
> The Team page shows which projects will sync. Exclude sensitive repositories before the first push. Leaving the team with `fleetlens team leave` stops future syncs.

What the server needs

- A Postgres database for team membership, daily rollups, usage history, integrations, and server-side reports.
- A stable FLEETLENS_ENCRYPTION_KEY for protected server-side credentials and tokens.
- A public HTTPS URL for browser sign-in and CLI pairing. Railway and Google Cloud setup paths provide one automatically.

7. Configuration and source builds

Most users do not need configuration. These are the supported knobs when the defaults do not fit.

Setting	Default	Use it when
CCLENS_PORT	3321	The default port is already in use
CCLENS_HOME	`~/.cclens`	You need isolated local state for another workspace or test run
GROK_HOME	`~/.grok`	Grok Build stores sessions or auth somewhere else
NEXT_OUTPUT	unset	You are building the web app for the bundled CLI

CHOOSE ANOTHER PORT

```
fleetlens start --port 4400
# or
CCLENS_PORT=4400 fleetlens start
```

Build from source

REPOSITORY CHECKOUT

```
git clone https://github.com/cowcow02/fleetlens.git
cd fleetlens
pnpm install
NEXT_OUTPUT=standalone pnpm build
node scripts/prepare-cli.mjs
node packages/cli/dist/index.js start
```

The monorepo builds parser, entries, web, team-server, and CLI packages through Turborepo. For the published experience, the CLI package contains the standalone dashboard bundle.

> **Versioning rule for contributors**
> The root package.json is the version source of truth. Use npm version at the repository root so sub-package versions stay synchronized; do not edit package versions one by one.

8. Troubleshooting

Use the symptom first, then run the smallest check that confirms the cause.

Symptom	Check	Next action
No sessions	Confirm an agent has local transcript files; check the source path for that agent	Run another agent session, then restart or refresh Fleetlens
Server will not start	`fleetlens status` and the port in the error	`Use fleetlens start --port 4400 or stop the old process`
No plan usage	`fleetlens daemon status; fleetlens daemon logs`	Log in to the provider CLI and wait for the next poll
UI looks stale after update	`fleetlens status; compare the serving version`	`Run fleetlens update, which restarts stale services cleanly`
Team is not syncing	`fleetlens team status` and `fleetlens team logs`	`Finish /team/onboarding, select projects, then run fleetlens team sync`
Digests are empty	Check entries in ~/.cclens and AI features in Settings	Run a session, enable AI features, or backfill recent days

Safe reset points

- Stopping Fleetlens does not delete transcript history or the local state directory.
- The local server and daemon are independent processes; fleetlens stop manages both, while fleetlens daemon stop only stops the worker.
- If a Team Edition server is unavailable, the local dashboard and local analytics continue to work. The daemon queues team payloads for retry where supported.

> **Still stuck?**
> Capture the output of `fleetlens status`, `fleetlens daemon logs`, and the exact command you ran. Remove tokens and private paths before sharing a log in a public issue.

9. Quick reference

The commands most users need after the first run.

Command	Purpose
`fleetlens start [--open]`	Start dashboard and usage daemon
`fleetlens stop`	Stop dashboard and usage daemon
`fleetlens status`	Show server, daemon, and latest snapshot
`fleetlens update`	Update to the latest published CLI
`fleetlens web [page] [--open]`	Open a dashboard page without starting the daemon
`fleetlens usage`	Print a current plan-utilization snapshot
`fleetlens usage --history`	Print daily token and cost history
`fleetlens entries --all`	Inspect day-scoped perception entries
`fleetlens digest day --yesterday`	Generate or read a day digest
`fleetlens digest week --last-week`	Generate or read a weekly digest
`fleetlens daemon logs`	Show recent daemon and sync logs
`fleetlens team status`	Show Team Edition pairing state

Where to go next

- Read the public platform documentation in the Fleetlens GitHub Pages site for architecture, data flow, deployment, and contributor notes.
- Open the repository README for release status, deployment links, and source-build details.
- Use the in-app Changelog icon to see user-facing changes in the installed build.

> **The one-line mental model**
> Agent transcripts are local inputs. The parser turns them into common session data. Analytics and entries turn that data into useful views. The local dashboard reads those results; Team Edition receives only the rollups you choose to share.

Fleetlens 0.16.4 | reviewed 2026-07-13